

Robot Operating System

Installation:-

We will be using ROS2 Humble distro.

It is encouraged to follow the official documentation [ROS Humble Setup](#) for installation.

ROS2 installation check:-

To check if ros is properly installed and configured run:-

```
Shell  
which ros2
```

If installed properly it should print `/opt/ros/humble/bin/ros2`

Prerequisite:

Before building ROS 2 packages, ensure that colcon is installed. Most ROS 2 development environments include it by default, but it can be installed manually using:

```
Shell  
sudo apt install python3-colcon-common-extensions -y
```

Verify the installation using:

```
Shell  
colcon --version
```

Nodes:-

Nodes are the fundamental building blocks of ROS 2. A node is an executable process that performs a specific task inside a robotic system. Instead of writing one huge program for the entire robot, ROS 2 divides functionalities into multiple independent nodes, making the system modular and easier to manage. For example:

- a. One node can control robot movement
- b. Another node can process camera data
- c. Another node can handle sensor readings

These nodes communicate with each other using topics, services, and actions. ROS 2 follows a distributed architecture where multiple nodes work together to achieve complex robotic tasks efficiently.

Run the below commands. These are pre built scripts that come with ROS2.

Shell

```
ros2 run demo_nodes_cpp talker
```

Shell

```
ros2 run demo_nodes_py listener
```

`demo_nodes_cpp` and `demo_nodes_py` are ROS 2 packages that contain different executable nodes. In this example, the `talker` node publishes messages while the `listener` node subscribes to those messages and displays them on the terminal.

- `talker` → sends messages on a topic
- `listener` → receives messages from that topic

This demonstrates the basic communication between ROS2 nodes using topics.

```
[INFO] [1778389679.472314200] [talker]: Publishing: 'Hello World: 491'
[INFO] [1778389680.472281165] [talker]: Publishing: 'Hello World: 492'
[INFO] [1778389681.472309166] [talker]: Publishing: 'Hello World: 493'
[INFO] [1778389682.472146070] [talker]: Publishing: 'Hello World: 494'
[INFO] [1778389683.472280435] [talker]: Publishing: 'Hello World: 495'
[INFO] [1778389684.472319983] [talker]: Publishing: 'Hello World: 496'
[INFO] [1778389685.472122084] [talker]: Publishing: 'Hello World: 497'
[INFO] [1778389686.472101979] [talker]: Publishing: 'Hello World: 498'
[INFO] [1778389687.472290217] [talker]: Publishing: 'Hello World: 499'
[INFO] [1778389688.472198339] [talker]: Publishing: 'Hello World: 500'
[INFO] [1778389689.472237641] [talker]: Publishing: 'Hello World: 501'
[INFO] [1778389690.472143119] [talker]: Publishing: 'Hello World: 502'
[INFO] [1778389691.472154080] [talker]: Publishing: 'Hello World: 503'
[INFO] [1778389692.472107508] [talker]: Publishing: 'Hello World: 504'
[INFO] [1778389693.472057118] [talker]: Publishing: 'Hello World: 505'
[INFO] [1778389694.472199244] [talker]: Publishing: 'Hello World: 506'
[INFO] [1778389695.472039600] [talker]: Publishing: 'Hello World: 507'
[INFO] [1778389696.472037781] [talker]: Publishing: 'Hello World: 508'
[INFO] [1778389697.472150758] [talker]: Publishing: 'Hello World: 509'
[INFO] [1778389698.472119160] [talker]: Publishing: 'Hello World: 510'
[INFO] [1778389699.472174179] [talker]: Publishing: 'Hello World: 511'
[INFO] [1778389700.472103079] [talker]: Publishing: 'Hello World: 512'
[INFO] [1778389701.472125740] [talker]: Publishing: 'Hello World: 513'
```

```
[INFO] [1778389698.473778221] [listener]: I heard: [Hello World: 510]
[INFO] [1778389699.474629472] [listener]: I heard: [Hello World: 511]
[INFO] [1778389700.473676768] [listener]: I heard: [Hello World: 512]
[INFO] [1778389701.474102268] [listener]: I heard: [Hello World: 513]
[INFO] [1778389702.473772627] [listener]: I heard: [Hello World: 514]
[INFO] [1778389703.473931731] [listener]: I heard: [Hello World: 515]
[INFO] [1778389704.473566317] [listener]: I heard: [Hello World: 516]
[INFO] [1778389705.473952808] [listener]: I heard: [Hello World: 517]
[INFO] [1778389706.473642307] [listener]: I heard: [Hello World: 518]
[INFO] [1778389707.473603719] [listener]: I heard: [Hello World: 519]
[INFO] [1778389708.473664839] [listener]: I heard: [Hello World: 520]
[INFO] [1778389709.473960247] [listener]: I heard: [Hello World: 521]
[INFO] [1778389710.473631286] [listener]: I heard: [Hello World: 522]
[INFO] [1778389711.473518912] [listener]: I heard: [Hello World: 523]
[INFO] [1778389712.473639211] [listener]: I heard: [Hello World: 524]
[INFO] [1778389713.473742000] [listener]: I heard: [Hello World: 525]
[INFO] [1778389714.473759655] [listener]: I heard: [Hello World: 526]
[INFO] [1778389715.473441338] [listener]: I heard: [Hello World: 527]
[INFO] [1778389716.473146020] [listener]: I heard: [Hello World: 528]
[INFO] [1778389717.472870745] [listener]: I heard: [Hello World: 529]
[INFO] [1778389718.473761412] [listener]: I heard: [Hello World: 530]
[INFO] [1778389719.473804873] [listener]: I heard: [Hello World: 531]
[INFO] [1778389720.473714173] [listener]: I heard: [Hello World: 532]
```

Publisher:-

A publisher is a ROS 2 node that sends data or messages to a topic. Other nodes that are interested in that data can subscribe to the same topic and receive the messages. For example, the talker node acts as a publisher by continuously publishing messages like "Hello World" on a topic, while the listener node receives them. Publishers are commonly used to send:

- Sensor data
- Camera images
- Robot velocity commands
- Status information

In ROS 2, publishers enable communication between different nodes in a distributed robotic system.

Subscriber:-

A subscriber is a ROS 2 node responsible for receiving messages published to a specific topic. It listens passively to a topic and processes any incoming data. Continuing the example, the **listener** node functions as a subscriber, consuming the messages published by the **talker** node. Subscribers are commonly used to gather information such as:

- Processed sensor data
- Received commands (e.g., movement goals)
- System notifications or alarms
- Current robot pose or state

This mechanism allows different parts of the robot system to access the information they need without being directly connected to the node that produced it.

Topics:-

Topics are the primary means of communication in ROS 2, functioning as named channels through which nodes exchange data messages. They use a publish/subscribe model where one node (the publisher) sends information to a topic, and any number of other nodes (subscribers) can simultaneously receive that information by subscribing to the same topic. Key characteristics of a Topic:

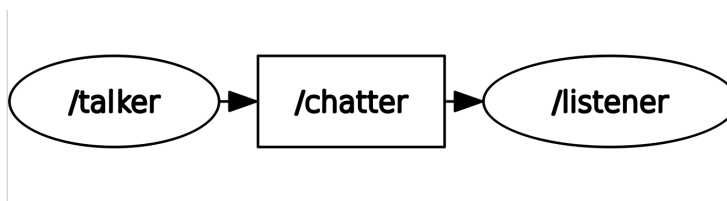
- **Communication Channel** A topic is essentially a continuous stream of data flowing through the robotic system.
- **Decoupling** Publishers and subscribers are decoupled. A publisher doesn't know how many subscribers are listening, and a subscriber doesn't know how many publishers are sending data, making the system flexible and modular.
- **Data Flow** Topics are used for data streams that update continuously and asynchronously. Examples from the system's operation include continuous transmission of:
 - Sensor data (like IMU orientation)
 - Robot state (like current robot pose or foot locations)
 - Commands (like joint angles sent to a controller)

The examples in the document demonstrate this: the **talker** node publishes messages to a topic, and the **listener** node subscribes to that same topic to receive and display the messages.

To view connection between multiple nodes in a ros program, run the following command

```
Shell
sudo apt update
sudo apt install '~nros-humble-rqt*'
rqt_graph
```

It opens a graph showing connection between different publisher, subscribers and the topics subscribed.



Here **/talker** is the publisher that publishes topic named **/chatter**. **/listener** is the subscriber that subscribes to the topic.

Turtlesim:-

Turtlesim is a simple, lightweight simulator included with ROS 2 that is commonly used to teach the core concepts of the system. It simulates a virtual turtle robot moving in a 2D environment, allowing users to interact with it using ROS 2 commands, nodes, and topics.

Shell

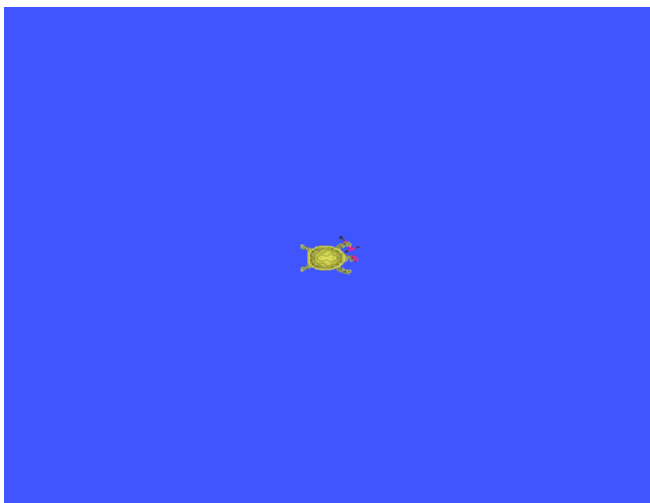
```
sudo apt install ros-humble-turtlesim
```

To run the Turtlesim node, use the following command:

Shell

```
ros2 run turtlesim turtlesim_node
```

Once running, the Turtlesim window will display a turtle that is ready to receive commands from other ROS 2 nodes. To control the turtle, another node (usually a teleoperation node) must publish velocity commands to the turtle's command topic.



Shell

```
ros2 run turtlesim turtle_teleop_key
```

Use teleop key to control motion of the turtle.

You can also give direct velocity command in the terminal without the turtle_teleop_key. It is like there is no publisher but topics are still enlisted so subscriber gets the topic. In reality the CLI behaves as the publisher. For e.g. the following command will move the turtle in circle.

Shell

```
ros2 topic pub /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear:
{x: 2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 1.8}}"
```

RQT:-

RQT is a ROS 2 Qt-based framework for GUI development. It provides a graphical user interface for visualizing, monitoring, and debugging various ROS 2 components through various plugins. One of its most commonly used tools is **rqt_graph**, which provides a visual representation of the ROS 2 computation graph, showing the connections between nodes, topics, and services.

You can see the nodes, and their associated topics, services, and actions, using the `list` subcommands of the respective commands:

Shell

```
ros2 node list
ros2 topic list
ros2 service list
ros2 action list
```

Services:-

Services enable communication between nodes using a request-response model. Unlike topics, services respond only when called.

Common Commands

These commands are useful for inspecting and managing active services:

Shell

```
ros2 service list # Lists all active services.
```

```
ros2 service type /clear # Shows the type of a service.
ros2 service list -t # Lists services along with their types.
ros2 service find std_srvs/srv/Empty # Finds services of a specific
type.
ros2 interface show turtlesim/srv/Spawn # Displays request and
response structure of a service.
```

Calling a Service

This demonstrates how to manually call a service from the command line:

```
Shell
ros2 service call /clear std_srvs/srv/Empty # Clears the turtlesim
screen.
ros2 service call /spawn turtlesim/srv/Spawn "{x: 2, y: 2, theta:
0.2, name: ''}" # Spawns a new turtle in turtlesim.
```

Actions:-

Actions are used for long-running tasks that require feedback and can be canceled. They consist of:

- Goal
- Feedback
- Result

Example: rotating a turtle in turtlesim. Actions are commonly used in robotics for tasks like navigation and motion control.

Common Commands

```
Shell
ros2 action list # Lists all active actions.
ros2 action list -t # Lists actions with their types.
ros2 action info /turtle1/rotate_absolute # Shows action clients
and servers.
ros2 interface show turtlesim/action/RotateAbsolute # Displays
goal, result, and feedback structure.
```

Sending an Action Goal

Shell

```
ros2 action send_goal /turtle1/rotate_absolute  
turtlesim/action/RotateAbsolute "{theta: 1.57}" # Rotates the  
turtle to a specified angle.
```

To receive continuous feedback:

Shell

```
ros2 action send_goal /turtle1/rotate_absolute  
turtlesim/action/RotateAbsolute "{theta: -1.57}" --feedback
```

Setting Up ROS Workspace

A ROS workspace is a directory with a particular structure. Commonly there is a `src` subdirectory. Inside that subdirectory is where the source code of ROS packages will be located. Typically the directory starts otherwise empty.

Introduction to ROS 2 Packages

Before writing ROS nodes, it is important to understand how ROS 2 packages are structured and managed. ROS 2 provides multiple build systems, with the two most common being:

- **ament_cmake** – Uses CMake as the build system and is traditionally associated with C++ development. However, it also fully supports Python nodes and is widely used in robotics projects.
- **ament_python** – Uses Python packaging tools and is designed specifically for pure Python ROS applications.

Both package types ultimately serve the same purpose: organizing ROS nodes and resources so they can be built, installed, and executed within the ROS ecosystem.

We will primarily use `ament_cmake` packages. One of the major reasons is that Python scripts can be executed seamlessly within an `ament_cmake` package by installing them through the `CMakeLists.txt` file. This allows us to write ROS nodes in Python while continuing to use the familiar CMake-based package structure.

In the following sections, we will explore how to create, build, and run both CMake-based and Python-based ROS 2 packages, along with their directory structures, important files, and recommended use cases.

Creating a CMake-Based ROS 2 Package (Recommended)

Go Inside the src Directory

Shell

```
cd ~/ros2_ws/src
```

Create a CMake Package

Shell

```
ros2 pkg create --build-type ament_cmake my_package
```

Explanation

- `ros2 pkg create` → Creates a new ROS package
- `ament_cmake` → Specifies that the package uses CMake as the build system
- `my_package` → Name of the package

Package Structure

After creation:

None

```
my_package/  
├─ CMakeLists.txt  
├─ package.xml  
├─ include/  
└─ src/
```

For Python-based ROS nodes using the CMake approach, many developers use:

None

```
my_package/  
├─ launch/  
├─ scripts/  
├─ srv/  
├─ CMakeLists.txt  
└─ package.xml
```

This is similar to the structure used in many ROS coursework assignments.

Important Files

package.xml

Contains:

- Package name
- Version
- Dependencies

- Maintainer information
- License information

ROS uses this file to manage package dependencies.

CMakeLists.txt

Very important for CMake packages.

It defines:

- Dependencies to be linked
- Installation rules
- Service and message generation
- Executable registration

ROS uses this file during the build process.

Example:

```
None
install(PROGRAMS
  scripts/my_first_node.py
  DESTINATION lib/${PROJECT_NAME}
)
```

This makes the Python script executable through ROS.

Writing Python Nodes

Create a scripts directory:

```
Shell
mkdir scripts
```

Create a Python node:

```
Shell
cd scripts
touch my_first_node.py
chmod +x my_first_node.py
```

Explanation

- `touch` → Creates the file
- `chmod +x` → Makes it executable

Example structure:

```
None
my_package/
```

```
|— scripts/  
|   └─ my_first_node.py  
|— CMakeLists.txt  
└─ package.xml
```

Installing Python Nodes

Add the following to CMakeLists.txt:

```
None  
  
install(PROGRAMS  
  scripts/my_first_node.py  
  DESTINATION lib/${PROJECT_NAME}  
)
```

Without this installation rule, ROS cannot run the node using `ros2 run`.

Complete Workflow

Typical ROS 2 CMake workflow:

1. Create workspace
2. Create src folder
3. Create CMake package
4. Create scripts folder
5. Write Python nodes
6. Add installation rules in `CMakeLists.txt`
7. Build workspace using `colcon build`
8. Source the workspace
9. Run ROS nodes

ROS 2 follows a modular structure where multiple nodes communicate using topics, services, actions, and parameters to perform robotic tasks efficiently.

Creating a Python Package

Go inside the src directory:

```
Shell  
  
cd ~/ros2_ws/src
```

Create a Python package:

```
Shell  
  
ros2 pkg create --build-type ament_python my_package
```

Explanation

- `ros2 pkg create` → Creates a new ROS package
- `ament_python` → Specifies that the package uses Python
- `my_package` → Name of the package

Package Structure

After creation:

```
None
my_package/
├─ package.xml
├─ setup.py
├─ setup.cfg
├─ resource/
├─ test/
└─ my_package/
    └─ __init__.py
```

Important Files

package.xml Contains:

- Package name
- Version
- Dependencies
- Maintainer information

ROS uses this file to manage dependencies between packages.

setup.py

Very important for Python packages. It defines:

- Python executable scripts
- Entry points for ROS nodes
- Installation details

ROS uses this file to identify runnable Python nodes.

__init__.py

Makes the folder behave as a Python module/package.

Writing Python Nodes

Go inside the package folder:

```
Shell
cd ~/ros2_ws/src/my_package/my_package
```

Create a Python node:

Shell

```
touch my_first_node.py
chmod +x my_first_node.py
```

Explanation

- touch → Creates the file
- chmod +x → Makes it executable

All ROS Python nodes will be written inside this directory. Example:

None

```
my_package/
├── my_package/
│   ├── __init__.py
│   └── my_first_node.py
```

Complete Workflow

Typical ROS 2 Python workflow:

- d. Create workspace
- e. Create src folder
- f. Create Python package
- g. Write Python nodes
- h. Build workspace using colcon build
- i. Source the workspace
- j. Run ROS nodes

ROS 2 follows a modular structure where multiple Python nodes communicate using topics, services, and actions to perform robotic tasks efficiently.

colcon performs out-of-source builds. By default it will create the /build, /install, /log directories as peers of the src directory:

Building the Workspace

Go back to the workspace root:

Shell

```
cd ~/ros2_ws
```

Build all packages:

```
Shell  
colcon build
```

What Happens During Build?

- ROS scans all packages inside src
- Dependencies are checked
- Python nodes are registered
- Executable files are generated
- build, install, and log folders are created

Sourcing the Workspace

After building:

```
Shell  
source install/setup.bash
```

Why Source?

This command informs the terminal about:

- Newly created packages
- ROS executables
- Environment variables

Without sourcing, ROS cannot detect your package.

Automating Source Command

To avoid sourcing every time:

Open .bashrc:

```
Shell  
gedit ~/.bashrc
```

Add:

```
Shell  
source ~/ros2_ws/install/setup.bash
```

Save and apply changes:

Shell

```
source ~/.bashrc
```

Running a Python Node

General syntax:

Shell

```
ros2 run <package_name> <node_name>
```

Example:

Shell

```
ros2 run my_package my_first_node
```

Resources:-

[ROS Humble Documentation](#) 👉 Follow the documentation for complete ROS 2 tutorial.

[ROS 2 Tutorial - YouTube Playlist](#) 👉 Watch complete playlist for better understanding.

[ROS OneShot](#) 👉 Up to timestamp 50:00.

[Turtlesim Tutorial](#) 👉 To understand turtlesim.

[Link](#) and [Link](#) 👉 Follow this for more info on services.